Классы. Объектно-ориентированный подход

Лекция 4

Класс - Это формальная модель или шаблон, задающий структуру объекта. Объект - это ещё одна сущьность, которая создаётся на основе класса (*штампуется на основе класса*). Между классом и объектом такие же отношения, как между чертежом и изделием на его основе. Например, есть чертеж детали, по которому ее изготавливают. Очевидно, что на основе чертежа можно изготавить мнжество детелей. Поэтом, важно корректно проработать сам чертёж. Ошибки чертежа в конечном итоге скажутся на ошибках в каждой детали.

Для объектно-ориентироанно метадологии важным является три принцыпа (основа): инкапсуляция, наследования, полиморфизм.

Под инкапсуляцией понимается сокрытие деталей реализации, данных и т.п. от внешней стороны.

Под наследованием понимается возможность создания нового класса на базе существующего.

Наследование предполагает наличие отношения “является” между классом наследником и классом родителем. ри этом класс потомок будет содержать те же атрибуты и методы, что и базовый класс, но при этом его можно (и нужно, раз создаётся новый класса) расширять через добавление новых методов и атрибутов.

Полиморфизм позволяет одинаково обращаться с объектами, имеющими однотипный интерфейс, независимо от внутренней реализации объекта. Например, с объектом класса "грузовой автомобиль" можно производить те же операции, что и с объектом класса "автомобиль", т.к. первый является наследником второго, при этом обратное утверждение неверно (во всяком случае не всегда). Другими словами полиморфизм **предполагает разную реализацию методов с одинаковыми именами**. Это очень полезно при наследовании, когда в классе наследнике можно переопределить методы класса родителя.

Создадим класс Точка (PointXY) в декартовой системе координат

In [1]:
```
class PointXY:
    color = "red"
    X = 0.0
    Y = 0.0
```

Класс состоит из объявления (инструкция class), имени класса (PointXY) и тела класса, которое содержит атрибуты и методы (пока только три атрибута).

Атрибуты - это набор данных, характеризующих объект или его состояние.

Создадим объкт нового класса. Для того чтобы создать объект класса необходимо воспользоваться следующим синтаксисом:

имя_объекта = имя_класса()

In [2]:
```
point_Z = PointXY()
```

Обратимся к атрибуту объекта

In [14]:
```
print(' Наша точка:(', point_Z.X, ',', point_Z.Y, ')')
```

```
 наша точка:( 0.0 , 0.0 )
```

Методы и атрибуты класса

Добавим к нашему классу **метод**. Метод – это функция, находящаяся внутри класса и выполняющая определенную работу.

In [4]:
```
class PointXY:
    color = "red"
    def __init__(self, x, y):
        self.x = x
        self.y = y
```

Объявлением: __init__(self, x, y) класса создает создаётся специальный метод - инициализатор. Этот метод вызывается всякий раз, когда нужно проинициализировать атрибуты для объекта (записать в них значения). Метод __init__ вызывается единожды, и не может быть вызван снова внутри программы для этого же объекта.
Как и в других языках программирования, так же существуют конструкторы. Конструктор - это специальный метод, который называется __new__. Только в Python конструктор существует и работает «за кулисами». В качестве единственного параметра он принимает класс, анализирует его код и на базе этой структуры создает пустой объект. Затем объетом занимается инициализатор. **Оба эти метода вызываются автоматически, когда создаётся объект: сначала __new__, потом __init__ (конструктор, инициализатор).**

In [5]:
```
A = PointXY(4,5)
B = PointXY(-6, 3.7)
```

синтаксис языка позволяет создать только один инициализатор, но его можно реализвать с переменным количеством параметров

In [27]:
```
# класс
class PointXY:
    color = "red"
    def __init__(self, *params):
        if len(params) > 0:
            self.x = params[0]
        else:
            self.x = 0.0

        if len(params) > 1:
            self.y = params[1]
        else:
            self.y = 0.0

        if len(params) > 2:
            self.type = params[2]
        else:
```

```
            self.type = 'OXY0' if (self.y >= 0.0) else 'OXY1'

# создадим объекты
C0 = PointXY()
C1 = PointXY(1.0)
C2 = PointXY(1.0, -1.5)
C3 = PointXY(1.0, -1.5, 'OXY1')

# распечатаем точку
print(f' ({C0.x}, {C0.y}) тип: {C0.type}')
print(f' ({C1.x}, {C1.y}) тип: {C1.type}')
print(f' ({C2.x}, {C2.y}) тип: {C2.type}')
print(f' ({C3.x}, {C3.y}) тип: {C3.type}')
```

```
 (0.0, 0.0) тип: OXY0
 (1.0, 0.0) тип: OXY0
 (1.0, -1.5) тип: OXY1
 (1.0, -1.5) тип: OXY1
```

Чтобы получить имя класса: Комбинации функции type() и __name__ для получения имени класса Комбинации __class__ и __name__ для имени класса

In [28]:
```
print(C0.__class__.__name__)
```

```
PointXY
```

Статические и динамические атрибуты класса

Атрибут может быть статическим (уровня класса) и динамическим (уровня объекта класса). Для работы со статическим атрибутом: не нужно создавать экземпляр класса, а для работы с динамическим – нужно. Динамический – характеризует конкретный объект, статический – класс.

В нашем классе, атрибут color – это статический атрибут, и доступ к нему, можно получить не создавая объект класса PointXY. Но, можно обратиться к этому атрибуту и через объект (плохой тон). Если обратиться к статическому атрибуту через объект, то у данного экземпляра будет создан динамический атрибут с таким же именем как статический, а доступ к последнему будет потерян. Доступ к динамическому атрибуту можно получить только черех объект класса, так как этот атрибут характризует/принадлежит конкретному его экземпляру.

In [29]:
```
print(PointXY.color, ' ', B.color)
```

```
red   red
```

Можно изменить статический атрибут, но для объекта *B*, уже создан одноименный атрибут и связь со статическим потеряна

In [39]:
```
PointXY.color = "green"
```

In [41]:
```
print(A.color, ' ', B.color, ' ', PointXY.color)
```

```
green   red   green
```

Теперь и для объекта *A* также потеряна связь со статическим атрибутом.

Как правило с атрибутами работают не напрямую, а через свойства

Замечание! Следует заметить что внутри класса к статическим атрибутам мы не обращаемся через self. Вне класса мы обращаемся к статическим атрибутам не через <имя объекта> с точкой, а через <Имя класса> с точкой (так как статические атрибуты характеризуют класс). То Внутри обычного метода доступ к статического атрибуту также следует организовать используя: имя_класса.имя_атрибута.

Статические атрибуты применяются для того, чтобы иметь одну общую переменную для всех объектов класса.

Дело в том, что при создании новых объектов создаются копии всех динамических атрибутов. Статические атрибуты относятся не к объекту, а к классу и имеют только одну копию.

Можно привести пример статического атрибута, который служить для подсчета количества созданных объектов класса.

Методы

Методы бывают **статическими**, **классовыми** и **уровня класса** (будем их называть просто словом метод).

Статический метод создается с декоратором @staticmethod,

классовый – с декоратором @classmethod, первым аргументом в него передается cls,

обычный метод создается без специального декоратора, ему первым аргументом передается self.
Каждый метод должен иметь как минимум один аргумент

In [39]:
```
class PointXY:
    color = "red"
    def __init__(self, x, y):
        self.x = x
        self.y = y

    @staticmethod #статический
    def static_method():
        print("\n static method \t атрибут:" + PointXY.color)

    @classmethod # уровня класса
    def class_method(cls):
        print(f'\n class method \t атрибут:{cls.color}')

    def write(self):# обычный метод
        print(f'\n обычный метод ({self.x}, {self.y})')
```

Статическийи классовый методы можно вызвать, не создавая экземпляр класса (название_класса.название_метода(...), для вызова обычного метода нужно обязательно создать объект: назвние_объекта.название_метода(...).

In [40]:
```
C = PointXY(2, -3) #создание объекта
C.write() #обычный метод
```

```
PointXY.class_method() #классовый метод
PointXY.static_method() #статический метод
```

```
 обычный метод (2, -3)

 class method    атрибут:red

 static method   атрибут:red
```

self - классам нужен, чтобы ссылаться на текущий объект. Ссылка *self* это способ указать любой объект.

Ститические и классовые методы не могут оратиться к динамическим атрибутам (только к статическим, так как не присутствует ссылка *self*)

Обычный метод (или метод экземпляра класс) - это наиболее часто используемый вид методов. Этот метод принимает объект класса: первый аргумент, который принято называть self и который указывает на объект. Количество параметров метода не ограничено.

In [57]:
```
class PointXY:
    color = "red"
    def __init__(self, x, y):
        self.x = x
        self.y = y
    def write(self, str):# обычный метод
        print(f'\n обычный метод ({self.x}, {self.y}) строка: {str}')
P = PointXY(3.5, -2.6)
P.write("обычный метод")
```

```
 обычный метод (3.5, -2.6) строка: обычный метод
```

Используя параметр self, можно менять состояние объекта и обращаться к другим его методам и параметрам. К тому же, используя атрибут self.__class__, можно получить доступ к атрибутам класса и возможности менять состояние самого класса. То есть методы экземпляров класса позволяют менять как состояние определённого объекта, так и класса.

Напечатаем название класса для построенного объекта, а также его методы и статические атрибуты (принадлежат классу)

In [95]:
```
class PointXY:
    color = "red"
    def __init__(self, x, y):
        self.x = x
        self.y = y
    def write(self, str):# обычный метод
        print(f'\n Обычный метод, атрибуты:({self.x}, {self.y}), строка: {str}')
        print(f'\n Класс: {self.__class__.__name__},\n атрибуты и методы класса {self._
P = PointXY(3.5, -2.6)
P.write("вывод")
```

```
 Обычный метод, атрибуты:(3.5, -2.6), строка: вывод

 Класс: PointXY,
 атрибуты и методы класса dict_keys(['__module__', 'color', '__init__', 'write', '__di
ct__', '__weakref__', '__doc__']), )
```

Пояснение.

weakref - специальный служебный атрибут, который содежит ссылку на класс

dict - словарь всех объявлений класса

module - название модуля, в котором объявлен класс

и т.д.

Классовый метод
Классовый метод принимают класс в качестве параметра, который принято обозначать: cls.
В нашем случае он указывает на класс PointXY, а не на объект этого класса.
Классовый метод привязаны к самому классу, а не его экземпляру. Они могут менять состояние класса, что отразится на всех объектах этого класса, но не может менять конкретный объект.

In [114…
```
class Student:
    counter = 0
    def __init__(self, f, balls = [0]):
        self.f = f
        self.balls = balls
        Student.counter += 1


    def write(self, str = "Студент"): # обычный метод
        print(f'{str}: {self.f}, {self.balls} \n')

    @classmethod # уровня класса
    def generate_stud(cls, params):
        print(f'\nСоздали нового студента с атрибутами:{params}')
        return cls(params[0],params[1])

    @classmethod # уровня класса
    def print_count_student(cls):
        print(f'\nКоличество студентов: {cls.counter}')


Stud = Student.generate_stud(["Сидоров", [10,23]])
Stud.write()
Stud.print_count_student()
```

```
Создали нового студента с атрибутами:['Сидоров', [10, 23]]
Студент: Сидоров, [10, 23]


Количество студентов: 1
```

Классовый метод используется тогда (чаще всего), когда нужен генерирующий метод, возвращающий объект класса.

Статический метод Статические методы нужны для выполнения монипуляций, которые напрямую не связаны ни с объектом ни с классом. Их можно воспринимать как методы, которые “не знают, к какому классу относятся”. Таким образом, статические методы прикреплены к классу лишь для удобства и не могут менять состояние ни класса, ни его экземпляра. Хотя, если статическому методу передать объект класса в качестве параметра, то можно будет изменить его атрибуты и/или вызвать его методы, если их не сделали закрытыми.

In [69]:
```
class Student:
```

```
    counter = 0
    def __init__(self, f, balls = [0]):
        self.f = f
        self.balls = balls
        Student.counter += 1

    def write(self, str = "Студент"): # обычный метод
        print(f'{str}: {self.f} баллы:{self.balls} \n')

    @staticmethod # статический метод
    def generate_stud(params):
        print(f'\nСоздали нового студента с атрибутами:{params}')
        return Student(params[0],params[1])

    @staticmethod # статический метод
    def print_count_student():
        print(f'\nКоличество студентов {Student.counter}')

    @staticmethod # статический метод
    def print(Stud):
        Stud.write()

Stud = Student.generate_stud(["Сидоров", [10,23]])
Stud.write()

Student.print_count_student()
Student.print(Stud)
```

```
Создали нового студента с атрибутами:['Сидоров', [10, 23]]
Студент: Сидоров баллы:[10, 23]


Количество студентов 1
Студент: Сидоров баллы:[10, 23]
```

Выводы:
Для объявления обычного метода декаратры не нужны, а вот классовый и статический методы объявляются с декараторами. Функционал, который можно реализовать Статические методы в основном используются как вспомогательные функции и работают с данными, которые им передаются.

Параметры self и cls — не обязательные названия, их можно называть иначе. Это лишь общепринятые обозначения, к которым привыкли разработчики. Но, они должны находиться первыми в списке параметров.

Перегрузка методов (функций)

Перегрузка методов (функций) далее делится на два типа: перегрузка встроенных функций и перегрузка пользовательских функций. Рассмотрим.

Перегрузка встроенных функций

Чтобы изменить поведение функции len () – это одна из встроенных функций.

Определилим для нашего класса специальный метод с именем *len* (). Каждый раз, когда передаем объект нашего класса в len(), результат будет получен путем вызова нашей пользовательской функции, то есть *len*() – которую создали для класса.

In [102…
```
import math

class PointXY:
    color = "red"
    def __init__(self, x, y):
        self.x = x
        self.y = y
    def write(self, str):# обычный метод
        print(f'\n обычный метод ({self.x}, {self.y}) строка: {str}')

    def __len__(self):# тоже обычный метод - перегрузка
        return int(math.sqrt(self.x**2 + self.y**2))

P = PointXY(3.5, -2.6)
print(len(P))
```

```
4
```

Замечание! Python ожидает, что функция len() вернет целое число, поэтому это следует учитывать при перегрузке функции. Если ожидается, что ваша перегруженная функция вернет не целое число, получите: TypeError.

Перегрузка пользовательских функций

В Python нельзя реаизовать общеизвестную перегрузку функций класса, как в С-подобных языках (когда, в классе объявляется несколько одноимённых функций с разным списком параметров). В Python нет возможно идентифицироват конкретный вариант такой функции, так как нельзя задать типы параметров для её вызова.
Чтобы перегрузить пользовательскую функцию в Python (иммитировать перегрузку), нужно написать логику функции таким образом, чтобы в зависимости от передаваемых параметров внутри функции выполнялся другой фрагмент кода.
Например:

In [60]:
```
class PointXY:
    color = "red"
    def __init__(self, x = 0.0, y = 0.0):
        self.x = x
        self.y = y
    def write(self, param = "строка", p = '\n'):# обычный метод
        if type(param)==type("string"):
            print(f'{p}  ({self.x}, {self.y}) параметр: {param} {p}')
        if type(param)==type(0):
             print(f'{p}  ({self.x + param}, {self.y + param}) параметр: {param} {p}')

P1 = PointXY()
P1.write("вывод", '\n')

P2 = PointXY(-3.8, -6.6)
P2.write(-4)
```

```
(0.0, 0.0) параметр: вывод


(-7.8, -10.6) параметр: -4
```

Лекция 5

Перегрузка операторов Можно назначить поведение оператора, который применяется к объектам созданного класса. Такая практика называется "перегрузкой оператора".

Функциональность операторов Python зависит от встроенных классов. Однако один и тот же оператор будет вести себя по-разному при применении к разным типам (классам). Например, оператор "+": выполняет арифметическую операцию при применении к двум числам, объединяет две строки, объединяет два списка. Для нового класса может также выполнять какой-то функционал.

Важно, чтобы алгоритмическая составляющая такой перегрузки логически соответствовала оператору!

Часто перегружаемые математические операторы

Название метода	Оператор
__add__(self, other)	+
__sub__(self, other)	–
__mul__(self, other)	*
__truediv__(self, other)	/
__mod__(self, other)	%
__lt__(self, other)	<
__le__(self, other)	<=
__eq__(self, other)	==
__ne__(self, other)	!=
__gt__(self, other)	>
__ge__(self, other)	>=

In [112…

```python
import math

class VectorXY:

    def __init__(self, x = 0.0, y = 0.0):
        self.x = x
        self.y = y
    def write(self):# обычный метод
        print(f'\nВектор ({self.x}, {self.y})')

    def __len__(self):# тоже обычный метод - перегрузка
        return int(math.sqrt(self.x**2 + self.y**2))

    def __str__(self):# тоже обычный метод - перегрузка
        return f'\nВектор ({self.x:.2f}, {self.y:.2f})'
```

```
    def __add__(self, B):
        return VectorXY(self.x+B.x, self.y+B.y)

    def __sub__(self, B):
        return VectorXY(self.x-B.x, self.y-B.y)

P1 = VectorXY(5.8, -4.2)
P2 = VectorXY(-8.3, 2.7)
print(P1, " \t", P2) # str(P1) str(P2)
print("\nСумма векторов: ", P1 + P2)
print("\nРазность векторов: ", P1 - P2)
```

```
Вектор (5.80, -4.20)
Вектор (-8.30, 2.70)

Сумма векторов:
Вектор (-2.50, -1.50)

Разность векторов:
Вектор (14.10, -6.90)
```

Задание свойств

Свойством называется такой метод класса, работа с которым подобна работе с атрибутом. Для объявления метода свойством необходимо использовать декоратор @property.

Важным преимуществом работы через свойства является то, что вы можете осуществлять проверку входных значений, перед тем как присвоить их атрибутам (помните про принцып: **инкапсуляции**, использование свойств как как позволяет скрывать внутренность класса, предоставляя интерфейс- свойства).

In [38]:
```
class Student:
    counter = 0
    def __init__(self, f, balls = [0]):
        self.f = f
        self.balls = balls
        Student.counter += 1

    @property
    def f(self):
        return self.__f.capitalize() # первый символ в верхний регистр
    @f.setter
    def f(self, a):
        if len(a) > 0:
            self.__f = a
        else:
            print("Error")

    @property
    def balls(self):
        return sorted(self.__balls) #сортирвка списка
    @balls.setter
    def balls(self, a):
        if min(a) >= 0:
            self.__balls = a
        else:
```

```
            print("Error list")

    def write(self, str = "Студент"):
        print(f'{str}: {self.f}, {self.balls} \n')

Stud = Student("иванов")
Stud.write()

Stud1 = Student("ПЕТРОВ", [10, 5, 78, 23])
Stud1.write()
# setter
Stud.balls = [4, 7, 89, 20]
Stud.f = "сидоров"
Stud.write()
```

```
Студент: Иванов, [0]

Студент: Петров, [5, 10, 23, 78]

Студент: Сидоров, [4, 7, 20, 89]
```

Можно не только читать (getter), но и задавать новые значения свойствам (@setter)
Свойство объявляется как метод без параметров (кроме self, естественно), декорированный с помощью property. Такой метод, возвращающий динамически вычисляемое значение, называется геттером (getter).
Сеттер — это тоже метод, который принимает новое значение для атрибута и как-то его обрабатывает.
Можно объявить только только getter. Только setter объявить нельзя.

In [23]:
```
class Student:
    counter = 0
    def __init__(self, f, balls = [0]):
        self.f = f
        self.balls = balls
        Student.counter += 1
 # getter и setter
    @property
    def f(self):
        return self.__f.capitalize()    # первый символ в верхний регистр
    @f.setter
    def f(self, a):
        if len(a) > 0:
            self.__f = a
        else:
            print("Error")
 # getter и setter
    @property
    def balls(self):
        return sorted(self.__balls)
    @balls.setter
    def balls(self, a):
        if min(a) >= 0:
            self.__balls = a
        else:
            print("Error list")
 # только getter
    @property
    def midlballs(self):
```

```
        return sum(self.balls)/len(self.balls) # средний балл

    def write(self, str = "Студент"):
        print(f'{str}: {self.f}, {self.balls} средний балл:{self.midlballs}\n')


Stud1 = Student("ПЕТРОВ", [10, 5, 78, 23])
Stud1.write()
# setter
Stud2 = Student("новенький")
Stud2.balls = [4, 7, 89, 20]
Stud2.f = "сидоров"
Stud2.write()
```

```
Студент: Петров, [5, 10, 23, 78] средний балл:29.0

Студент: Сидоров, [4, 7, 20, 89] средний балл:30.0
```

Замечание

Хорошим тоном считается, что для чтения/изменения какого-то атрибута должны использоваться:

специальные методы, которые называются getter/setter или

свойства, которые объявляются с декоратором @property.

Замечание

В большинстве языков с поддержкой ООП, можете явно указать для атрибута, что доступ к нему снаружи класса запрещен, это делается с помощью ключевых слов (private, protected и т.д.). В Python таких модификаторов (спецификаторов доступа) нет.

Реализация принципов ООП (инкапсуляция) должна предусмотреть возможность запретить прямой доступ к атрибуту (или к методу) со стороны внешнего кода. То есть сделать его скрытым, приватным. Прямая модификация некоторых атрибутов может привести к дефектам в программе. Нужно ведь оставить открытыми только те атрибуты и методам, которые будут взаимодействовать с внешним кодом (образуют интерфейс класса). Остальные атрибуты и функции, которые, например, обслуживают свой класс, не должны быть доступны извне.

В Python это реализовано с помощью добавления к имени атрибута или метода одинарного или двойного подчеркивания.

Первое используется в расчете на внимательных программистов.

Второе: более жесткий вариант для всех остальных.
Подробнне:

Если мы заменим balls на _balls, он станет скрытым атрибутом. Но к такому атрибуту еще можно получить доступ как обычно, через <имя объекта>._balls. Это делать не рекомендуется, но, в исключительных случаях, сознательные и внимательные программисты умеют без последствий обращаться к скрытым атрибутам.

Если заменим balls на __balls (в примере как раз было это использовано), он станет полностью закрытым атрибутом. И к нему уже нельзя будет получить доступ обычным способом. Интерпретатор языка Python выдаст ошибку AttributeError и не выполнит этот код.

In [73]:
```
# инкапсулируем атрибут класса и метод
class Student:
    __counter = 0
    def __init__(self, f, balls = [0]):
        self.__f = f
        self._balls = balls
        Student.__counter += 1

    def __write(self, str = "Студент"): # обычный метод
        print(f'{str}: {self.__f} баллы:{self._balls} \n')

    @staticmethod # статический метод
    def generate_stud(params):
        print(f'\nСоздали нового студента с атрибутами:{params}')
        return Student(params[0],params[1])

    @staticmethod # статический метод
    def print_count_student():
        print(f'\nКоличество студентов {Student.__counter}')

    @staticmethod # статический метод
    def print(Stud):
        Stud.__write()

Stud = Student.generate_stud(["Сидоров", [10,23]])
#Stud.write() #выдаёт ошибку -  метод закрытый
print(Stud._balls)
#print(Stud.__f) #выдаёт ошибку - атрибут закрытый
Student.print_count_student()
Student.print(Stud)
```

```
Создали нового студента с атрибутами:['Сидоров', [10, 23]]
[10, 23]

Количество студентов 1
Студент: Сидоров баллы:[10, 23]
```

Наследование

Наследование позволяет «экономить» код, время и силы, не делая лишних объявлений.

Для объяявления наследования нужен родительский класс (заранее должен быть создан) и класс потомок, для которого указавается родить. Возможно реализовать множественное наследование, в этом случае у класса потомка может быть несколько родителей. Не все языки программирования поддерживают множественное наследование, но в Python можно его использовать. Нужно помнить, что множественное наследование следует исползовать только в том случае, когда необходимо объеденить свойства и поведение нескольких классов в новом классе.

По умолчанию все классы в Python являются наследниками от object, хотя это не прописывается при создании нового класса (это происходит автоматически).

Синтаксически объявения класса наследника:

class имя_класса(родитель1, родитель2, родитель3 ...)

In [131…

```
# родитель
class Student:
    __counter = 0
    def __init__(self, f, balls = [0]):
        self.__f = f
        self.balls = balls
        Student.__counter += 1

    def write(self, str = "Студент"): # обычный метод
        print(f'{str}: {self.__f} баллы:{self.balls}')

    @staticmethod # статический метод
    def print_count_student():
        print(f'\nКоличество студентов {Student.__counter}')


# наследник
class Good_Student(Student):
    __counterG = 0

    def __init__(self, f, balls = [0], grant = 10.5):
        super().__init__(f, balls) # родительский инициализатор
        self.__grant = grant
        Good_Student.__counterG += 1


    def write(self, str = "Хороший Студент"): # обычный метод
        super().write("\nХороший Студент")
        print(f'стипендия:{self.__grant} \n')

    @staticmethod # статический метод
    def print_count_student():
        print(f'\nКоличество хороших студентов {Good_Student.__counterG}')
        Student.print_count_student() # родительский статический метод

Kira = Student("Иванова Кира", [10,36])
Kira.write()

Lena = Good_Student("Иванова Лена", [50,96])
Lena.write()

#статический метод, который печатает результаты и вызвает статический родительский мето
Good_Student.print_count_student()
```

```
Студент: Иванова Кира баллы:[10, 36]

Хороший Студент: Иванова Лена баллы:[50, 96]
стипендия:10.5


Количество хороших студентов 1

Количество студентов 2
```

Можно использовать две встроенные функции, работающие с наследованием:

Используйте isinstance() для проверки класса (типа) экземпляра: isinstance(obj, int) возвратит True, только если класс для объекта obj является int или некоторым классом полученным из int.

Используйте issubclass(), чтобы проверить наследственность класса: issubclass(bool, int) - True, т. к. bool - наследник класса int. Но, issubclass(float, int) - False, т. к. float не наследник int.

In [136…
```
print('issubclass(Good_Student, Student) :', issubclass(Good_Student, Student))
print('isinstance(Lena, Good_Student):', isinstance(Lena, Good_Student))
```

```
issubclass(Good_Student, Student) : True
isinstance(Lena, Good_Student): True
```

Полиморфизм проявляет себя с позиции переопределения методов базового класса в классе наследнике.

В примере объекты класса Good_Student получают другую реализацию метода write.

In [125…
```
Lena.write()
```

```
Хороший Студент: Иванова Лена баллы:[50, 96]
стипендия:10.5
```

Выводы:

Рассмотрены базовые вопросы построения классов и объектов, а также способы реализации остных принципов ООПрограммирования: инкапсуляция, наследование, полиморфизм.